

HMS MediCore HMS

Hospital Management System

Project Proposal

Instructor: Talha Shahid · FAST NUCES Karachi

Oracle XE 11.2 · Python Flask · HTML / CSS / JS

Team Members

Bismah Sheikh — 24K-0795

Filza Tanweer — 24K-0708

Fatima Haider — 24K-0551

1. Introduction

MediCore HMS is a proposed full-stack Hospital Management System designed to digitise and streamline the complete operational workflow of a modern hospital. The system is built on Oracle Database XE 11.2, a Python Flask REST API backend, and a responsive single-page web frontend that requires no external frameworks beyond a standard browser.

The project demonstrates advanced database concepts — normalisation to BCNF, stored procedures, BEFORE/AFTER triggers, concurrency control via SELECT FOR UPDATE, role-based access views, and ACID-compliant transactions — within a real-world, clinically meaningful application.

2. Problem Statement

Hospitals in Pakistan, particularly mid-sized facilities, continue to rely on manual paper-based or fragmented digital systems for patient management, pharmacy inventory, billing, and appointment scheduling. This leads to several critical operational problems:

- Double-booking of doctors and rooms due to lack of concurrency enforcement at the database level
- Delayed or missed low-stock alerts for critical medicines, risking patient care
- Inconsistent billing where invoices and insurance claims are managed in separate, disconnected records
- No role-based access control — clinical staff can view financial data and vice versa
- Patient discharge requiring manual updates across multiple registers with no atomicity guarantee

MediCore HMS addresses all of the above through a unified, database-enforced system where constraints, triggers, and stored procedures guarantee data integrity regardless of user behaviour.

3. Project Objectives

#	Objective	Priority
1	Design a fully normalised (BCNF) Oracle schema with 14 tables	High
2	Implement 5 stored procedures covering core business logic	High
3	Build 14 BEFORE INSERT triggers + 1 AFTER UPDATE stock alert trigger	High
4	Enforce RBAC using 7 database-level SQL views	High
5	Demonstrate pessimistic locking via SELECT FOR UPDATE	High
6	Implement optimistic concurrency via version stamp pattern	Medium
7	Build a complete REST API with 22 endpoints in Flask	High
8	Develop a single-page frontend with 13 functional modules	High
9	Add CSV export, dark mode, global search, and audit logging	Medium
10	Seed the database with realistic Pakistani hospital data	Medium

4. Proposed System Architecture

4.1 Technology Stack

Layer	Technology	Justification
Database	Oracle XE 11.2	Industry-standard RDBMS; full PL/SQL, triggers, stored procedures
Backend	Python 3 + Flask	Lightweight REST API; clean Oracle integration via python-oracledb
Frontend	HTML5 / CSS3 / JS	Single-file SPA — no build tools, runs in any modern browser
DB Driver	python-oracledb	Official Oracle driver; thick mode for full PL/SQL and trigger support
Deployment	Localhost	Oracle XE on Windows; Flask dev server on port 5000

4.2 System Modules

Module	Description
Dashboard	Live statistics, charts, quick actions, recent activity feed
Patient Management	Full CRUD, paginated registry, per-patient medical history timeline
Doctor & Departments	Staff roster with specializations, department heads, floor mapping
Appointments	Book/cancel, filter by status, inline updates, double-booking prevention
Room Availability	Live occupancy grid with capacity bars, color-coded room status
Medicine Inventory	Stock levels, minimum thresholds, supplier info, quick dispense
Lab Tests	Order management, result entry, and status tracking per patient
Invoices & Billing	ACID invoice creation, payment tracking, printable receipts

Insurance Claims	Multi-provider claim lifecycle tracking
Stock Alerts	Trigger-generated alerts with resolve workflow
Analytics	Charts: doctor workload, revenue trends, medicine usage
Audit Log	Client-side log of all INSERT/UPDATE operations
Concurrency Demo	Interactive SELECT FOR UPDATE simulation with two sessions

5. Proposed Database Design

5.1 Schema Overview — 14 Tables

All tables are designed to satisfy Boyce-Codd Normal Form (BCNF). Primary keys are assigned automatically via BEFORE INSERT triggers consuming Oracle sequences — replacing the PostgreSQL SERIAL pattern unsupported in Oracle XE 11.2.

Table	Group	Key Relationships & Constraints
patient	Core Entity	Referenced by appointment, invoice, lab_test (CASCADE DELETE); blood_group CHECK
supplier	Core Entity	Referenced by medicine_inventory; email UNIQUE; lead_time_days CHECK >= 0
department	Core Entity	Referenced by doctor, room, staff; circular FK head_doctor_id resolved via ALTER TABLE
doctor	FK-Dependent	References department; license_no UNIQUE; phone UNIQUE; referenced by appointment
medicine_inventory	FK-Dependent	References supplier; quantity CHECK >= 0; unit CHECK; referenced by alerts, presc_d
room	FK-Dependent	References department; occupancy <= capacity CHECK; referenced by appointment
staff	FK-Dependent	References department; role CHECK; shift CHECK
appointment	Operational Hub	4-way FK; UNIQUE(doctor_id, appt_datetime); UNIQUE(room_id, appt_datetime)
prescription	Weak Entity	Existentially dependent on appointment; UNIQUE(appointment_id); CASCADE DELETE
invoice	Weak Entity	Existentially dependent on patient; amount_paid <= total_amount CHECK
prescription_detail	Junction	Links prescription to medicine_inventory; UNIQUE(prescription_id, medicine_id)
lab_test	Detail	References patient, doctor, appointment; status CHECK; CASCADE DELETE on patient
insurance_claim	Detail	References invoice; UNIQUE(invoice_id) — one claim per invoice; resolved_at CHECK
alerts	System	Auto-populated by low_stock_alert trigger only — never inserted manually

5.2 Normalisation (BCNF)

- 1NF — All attributes are atomic; no repeating groups anywhere in the schema
- 2NF — No partial dependencies; all non-key attributes depend on the full primary key
- 3NF — No transitive dependencies; no non-key attribute determines another non-key attribute
- BCNF — Every determinant is a candidate key; stricter than 3NF and fully satisfied throughout

5.3 Sequences & Auto-Increment Strategy

Oracle XE 11.2 does not support IDENTITY columns (available only from Oracle 12c). Fourteen sequences are created — one per table — each with START WITH 1, INCREMENT BY 1, NOCACHE, NOCYCLE. A paired BEFORE INSERT trigger assigns the next value to the primary key column when NULL on insert.

6. Advanced Database Features

Triggers (15 total)	14 BEFORE INSERT triggers handle automatic primary key assignment via sequences and set SYSTIMESTAMP on created_at columns. One AFTER UPDATE trigger (low_stock_alert) fires automatically whenever quantity_available in medicine_inventory drops below minimum_threshold, inserting a descriptive record into the alerts table — demonstrating event-driven database automation.
Stored Procedures (5 total)	sp_book_appointment prevents double-booking with RAISE_APPLICATION_ERROR. sp_process_billing creates ACID invoices with SAVEPOINT-based partial rollback. sp_dispense_medicine validates and deducts stock (which triggers the alert). sp_resolve_alert validates and resolves stock alerts via SQL%ROWCOUNT. PROC_PATIENT_DISCHARGE atomically updates three tables — room, invoice, appointment.
RBAC Views (7 total)	Seven SQL views enforce role-based access at the Oracle database layer, not just the application: vw_doctor_view, vw_nurse_view, vw_accountant_view, vw_pharmacy_view expose role-specific data. v_doctor_roster, v_patient_privacy, and v_unpaid_invoices provide shared reference views.
Concurrency Control	Pessimistic locking via SELECT FOR UPDATE on the appointment table prevents double-booking race conditions. Optimistic concurrency via a version_stamp column handles lower-conflict patient record updates — both strategies are demonstrated live in the HMS frontend.
Performance Indexes	10 indexes on high-frequency query columns across appointment, invoice, lab_test, insurance_claim, medicine_inventory, and alerts tables. Oracle does not support partial indexes, so full indexes are created and filtering applied in WHERE clauses.
ACID Transactions	All write operations are wrapped in explicit transactions. PROC_PATIENT_DISCHARGE demonstrates full Atomicity — three table updates committed as a single unit, with complete ROLLBACK on any failure, satisfying Atomicity and Consistency.

7. Project Timeline

Phase	Deliverable	Status
Phase 1	Schema design, ER diagram, normalisation analysis	Completed
Phase 2	Oracle DDL — tables, sequences, constraints, seed data	Completed
Phase 3	Triggers, stored procedures, views, performance indexes	Completed
Phase 4	Advanced queries, concurrency control, ACID demonstrations	Completed
Phase 5	Flask REST API — all 22 endpoints	Completed

Phase 6	Frontend SPA — all 13 modules, dark mode, CSV export	Completed
Final	Testing, GitHub upload, documentation, final submission	Completed

8. Expected Outcomes

- ✓ A fully functional, browser-based Hospital Management System connected to a live Oracle XE database
- ✓ Demonstration of all major database concepts from CS-2005: normalisation, transactions, triggers, stored procedures, views, indexing, and concurrency
- ✓ A clean, documented GitHub repository containing schema SQL, backend Flask code, and frontend SPA
- ✓ Role-based access control enforced at the Oracle database layer — not merely at the application layer
- ✓ Automated business rules (stock alerts, patient discharge workflow) operating independently of the frontend
- ✓ An interactive concurrency demo visually proving double-booking prevention is enforced at the database level